

# Tools in LangChain & LangGraph

---

```
from IPython.display import Image
display(Image(filename="image/tool_ecosystem.png", width=400, height=50))
```



## Capabilities of an AI Agent

---

An **AI Agent** has two main capabilities:

### 1. Reasoning and Decision-Making Capability

The **LLM provides the reasoning and decision-making capability** of the agent.

The LLM can:

- Understand the user's request
- Analyze the problem
- Decide what needs to be done
- Select the appropriate tool
- Decide the next step based on the result

```
User Request
  ↓
  LLM
  ↓
Reasoning
  ↓
Decision
  ↓
Select Tool
```

---

### 2. Action Capability

**Tools provide the action capability** of the agent.

Tools allow the agent to interact with the outside world and perform tasks.

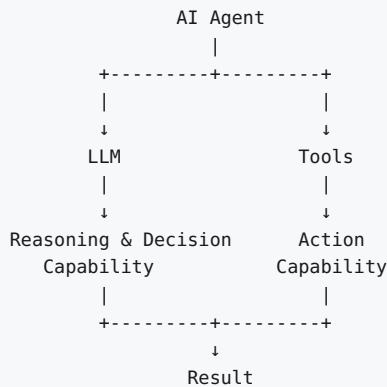
Examples:

- Search the web
- Query a database
- Call an API
- Perform calculations
- Read or write files
- Send an email

```
LLM Decision
  ↓
  Tool
  ↓
Perform Action
  ↓
Return Result
```

---

# Agent = LLM + Tools



## Key Point

The LLM provides the reasoning and decision-making capability, while tools provide the action capability.

## In Simple Words

LLM → Think & Decide  
Tools → Take Action

## What is a Tool?

A **tool** is simply a Python function, API, or external service that an LLM can use when it needs to perform a specific task.

Tools allow an AI application to interact with the outside world.

## Examples of Tools

- **Calculator** → Performs calculations
- **Search Engine** → Searches for information
- **Database** → Reads or updates data
- **Weather API** → Gets weather information
- **Email API** → Sends emails
- **Python Function** → Performs a custom task

<https://docs.langchain.com/oss/python/integrations/tools>

## Built-in Tools vs Custom Tools in LangChain

Tools in LangChain can generally be divided into two categories:

1. **Built-in Tools**
2. **Custom Tools**

### ✳ 1. Built-in Tools in LangChain

A **built-in tool** is a tool that LangChain already provides for common tasks.

Instead of creating the tool's function logic from scratch, we can simply **import the existing tool and use it**.

#### ✦ Examples of Built-in Tools

| Built-in Tool | Purpose |
|---------------|---------|
|---------------|---------|

|                      |                                   |
|----------------------|-----------------------------------|
| DuckDuckGoSearchRun  | 🔍 Search the web using DuckDuckGo |
| WikipediaQueryRun    | 📖 Get information from Wikipedia  |
| PythonREPLTool       | 🐍 Execute Python code             |
| ShellTool            | 📄 Run shell commands              |
| RequestsGetTool      | 🌐 Make HTTP GET requests          |
| GmailSendMessageTool | ✉️ Send emails using Gmail        |
| SlackSendMessageTool | 💬 Send messages to Slack          |
| SQLDatabaseQueryTool | 🗄️ Execute SQL queries            |

## Example

```
from langchain_community.tools import DuckDuckGoSearchRun

search_tool = DuckDuckGoSearchRun()
results = search_tool.invoke('top news in india today')
print(results)
```

Hindustan Times: Stay updated with Hindustan Times for the top India news, world events, and breaking stories. Get real-time updates on politics, wars, live cricket scores, entertainment... 8 minutes ago · Today's top India news headlines, news on Indian politics, elections, government, business, technology, and Bollywood. Check today's top 10 news stories, headline news, breaking news, latest news, politics news, sports news, entertainment news and business news on Times of India 18 minutes ago · Stay updated with the latest breaking news, live updates, photos, videos and top trending stories from India and around the world across politics, economy, sports and more on The Economic Times. Get the latest India news, breaking headlines, political developments, government updates, weather alerts, crime news and state coverage at CNBC TV18. Watch India Today Live TV for free – live stream breaking news, latest headlines, politics, sports, entertainment, world and english news streaming online at our Live TV coverage. Check today's top 10 news stories, headline news, breaking news, latest news, politics news, sports news, entertainment news and business news on Times of India 18 minutes ago · Stay updated with the latest breaking news, live updates, photos, videos and top trending stories from India and around the world across politics, economy, sports and more on The Economic Times. Get the latest India news, breaking headlines, political developments, government updates, weather alerts, crime news and state coverage at CNBC TV18.

```
print(search_tool.name) #Returns the tool name
print(search_tool.description) #Returns the tool description, usually from the docstring
print(search_tool.args) #Returns the input arguments/schema expected by the tool
```

```
duckduckgo_search
A wrapper around DuckDuckGo Search. Useful for when you need to answer questions about current events. Input should be a search query.
{'query': {'description': 'search query to look up', 'title': 'Query', 'type': 'string'}}
```

## 2. Custom Tools

**Custom tools** are tools that we create ourself for a specific application.

Usually, we create a Python function and convert it into a LangChain tool using the `@tool` decorator.

### Example

## Steps to Create a Tool Function

### Step 1 - Create a Function and Add a Docstring

Create a function and add a **docstring** that clearly explains the purpose of the function. The LLM uses the docstring to understand **what the function does and when it should be used**.

### Step 2 - Add Type Hints

Define **type hints** for the function's input parameters and output. This helps the LLM understand **what type of data the function expects and what it returns**.

### Step 3 - Add the `@tool` Decorator

Use the `@tool` decorator to convert the function into a tool. This tells the LLM that the function is **available as a tool that it can call when needed**.

```
from langchain_core.tools import tool
```

```
# Step 1 - create a function and add doc sting
```

```
def multiply(a, b):  
    """Multiply two numbers"""  
    return a*b
```

```
# Step 2 - add type hints
```

```
def multiply(a: int, b:int) -> int:  
    """Multiply two numbers"""  
    return a*b
```

```
# Step 3 - add tool decorator
```

```
@tool  
def multiply(a: int, b:int) -> int:  
    """Multiply two numbers"""  
    return a*b
```

```
result = multiply.invoke({"a":3, "b":5})  
result
```

```
15
```

```
print(multiply.name) #Returns the tool name  
print(multiply.description) #Returns the tool description, usually from the docstring  
print(multiply.args) #Returns the input arguments/schema expected by the tool
```

```
multiply  
Multiply two numbers  
{'a': {'title': 'A', 'type': 'integer'}, 'b': {'title': 'B', 'type': 'integer'}}
```

```
print(multiply.args_schema.model_json_schema())
```

```
{'description': 'Multiply two numbers', 'properties': {'a': {'title': 'A', 'type': 'integer'}, 'b': {'title':  
'B', 'type': 'integer'}}, 'required': ['a', 'b'], 'title': 'multiply', 'type': 'object'}
```

`print(multiply.args_schema.model_json_schema())` prints the **JSON Schema of the tool's input arguments**, This schema helps the LLM understand what arguments it needs to provide when calling multiply.

```

{
  'properties': {
    'a': {
      'title': 'A',
      'type': 'integer'
    },
    'b': {
      'title': 'B',
      'type': 'integer'
    }
  },
  'required': ['a', 'b'],
  'title': 'multiply',
  'type': 'object'
}

```

## There are several common ways to create a custom tool.

| Method         | Best for                              |
|----------------|---------------------------------------|
| @tool          | Simple Python functions               |
| StructuredTool | Custom input schemas and more control |
| BaseTool       | Advanced/custom tool behavior         |

### Method 2 - Structured Tool in LangChain

A **Structured Tool** in LangChain is a type of tool where the input follows a **structured schema**, typically defined using a **Pydantic model**.

The schema defines:

- The **input fields**
- The **data types**
- The **required fields**
- The **validation rules**

**Key point:** `StructuredTool` is useful when you need more control over the tool's **input structure and validation**.

```

from langchain_core.tools import StructuredTool
from pydantic import BaseModel, Field

```

```

class MultiplyInput(BaseModel):
    a: int = Field(required=True, description="The first number to add")
    b: int = Field(required=True, description="The second number to add")

```

```

/var/folders/yh/qy0zfnfx1rs0kdcl_cd8mrjh0000gn/T/ipykernel_14693/3279971488.py:2: PydanticDeprecatedSince20:
Using extra keyword arguments on `Field` is deprecated and will be removed. Use `json_schema_extra` instead.
(Extra keys: 'required'). Deprecated in Pydantic V2.0 to be removed in V3.0. See Pydantic V2 Migration Guide at
https://errors.pydantic.dev/2.13/migration/
  a: int = Field(required=True, description="The first number to add")
/var/folders/yh/qy0zfnfx1rs0kdcl_cd8mrjh0000gn/T/ipykernel_14693/3279971488.py:3: PydanticDeprecatedSince20:
Using extra keyword arguments on `Field` is deprecated and will be removed. Use `json_schema_extra` instead.
(Extra keys: 'required'). Deprecated in Pydantic V2.0 to be removed in V3.0. See Pydantic V2 Migration Guide at
https://errors.pydantic.dev/2.13/migration/
  b: int = Field(required=True, description="The second number to add")

```

```

def multiply_func(a: int, b: int) -> int:
    return a * b

```

```
multiply_tool = StructuredTool.from_function(
    func=multiply_func,
    name="multiply",
    description="Multiply two numbers",
    args_schema=MultiplyInput
)
```

```
result = multiply_tool.invoke({'a':3, 'b':3})

print(result)
print(multiply_tool.name)
print(multiply_tool.description)
print(multiply_tool.args)
```

```
9
multiply
Multiply two numbers
{'a': {'description': 'The first number to add', 'title': 'A', 'type': 'integer'}, 'b': {'description': 'The
second number to add', 'title': 'B', 'type': 'integer'}}
```

## Method 3 - Using `BaseTool` Class

`BaseTool` is the **base class for creating custom tools in LangChain**. It defines the core structure and interface that a tool should follow.

We can extend `BaseTool` when you need **full control over the tool's behavior, input handling, and execution logic**.

Other tool implementations, such as `StructuredTool`, follow the tool interface defined by LangChain's core tool abstractions.

### Simple Flow

**BaseTool** ↓ **Create a Custom Tool Class** ↓ **Define Tool Metadata + Execution Logic** ↓ **Tool can be used by the LLM**

### When to Use `BaseTool`

Use `BaseTool` when you need **advanced customization** beyond what `@tool` or `StructuredTool` provides.

```
from langchain.tools import BaseTool
from typing import Type
```

```
# arg schema using pydantic
```

```
class MultiplyInput(BaseModel):
    a: int = Field(required=True, description="The first number to add")
    b: int = Field(required=True, description="The second number to add")
```

```
/var/folders/yh/qy0zfnfx1rs0kdcl_cd8mrjh0000gn/T/ipykernel_14693/908171234.py:4: PydanticDeprecatedSince20: Using
extra keyword arguments on `Field` is deprecated and will be removed. Use `json_schema_extra` instead. (Extra
keys: 'required'). Deprecated in Pydantic V2.0 to be removed in V3.0. See Pydantic V2 Migration Guide at
https://errors.pydantic.dev/2.13/migration/
```

```
    a: int = Field(required=True, description="The first number to add")
/var/folders/yh/qy0zfnfx1rs0kdcl_cd8mrjh0000gn/T/ipykernel_14693/908171234.py:5: PydanticDeprecatedSince20: Using
extra keyword arguments on `Field` is deprecated and will be removed. Use `json_schema_extra` instead. (Extra
keys: 'required'). Deprecated in Pydantic V2.0 to be removed in V3.0. See Pydantic V2 Migration Guide at
https://errors.pydantic.dev/2.13/migration/
    b: int = Field(required=True, description="The second number to add")
```

```
class MultiplyTool(BaseTool):
    name: str = "multiply"
    description: str = "Multiply two numbers"

    args_schema: Type[BaseModel] = MultiplyInput

    def _run(self, a: int, b: int) -> int:
        return a * b
```

```
multiply_tool = MultiplyTool()
```

```
result = multiply_tool.invoke({'a':3, 'b':3})

print(result)
print(multiply_tool.name)
print(multiply_tool.description)

print(multiply_tool.args)
```

```
9
multiply
Multiply two numbers
{'a': {'description': 'The first number to add', 'title': 'A', 'type': 'integer'}, 'b': {'description': 'The
second number to add', 'title': 'B', 'type': 'integer'}}
```

## Toolkits in LangChain

A **Toolkit** is simply a **collection (bundle) of related tools** that work together for a **common purpose**.

Instead of creating and managing each tool individually, LangChain allows you to group related tools into a **Toolkit** for easier **reuse, organization, and integration**.

### Example: Google Drive Toolkit

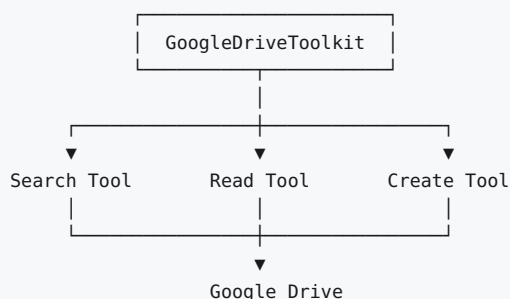
A toolkit could be:

**GoogleDriveToolkit**

It may contain multiple tools such as:

```
GoogleDriveToolkit
|
├─ 📁 Search Google Drive
├─ 📄 Read Google Drive File
├─ ➕ Create / Update File
├─ 📋 List Files
└─ 🗑 Delete File
```

### How it works



### Simple way to remember

**Tool = One capability Toolkit = Collection of related capabilities**

So, when an **agent needs to work with Google Drive**, we can provide the **whole toolkit**, giving the agent access to multiple related tools rather than adding each tool separately.

```

from langchain_core.tools import tool

@tool
def add(a: int, b: int) -> int:
    """Add two numbers"""
    return a + b

@tool
def multiply(a: int, b: int) -> int:
    """Multiply two numbers"""
    return a * b

class MathToolkit:
    def get_tools(self):
        return [add, multiply]

# Create toolkit
toolkit = MathToolkit()

# Get tools from toolkit
tools = toolkit.get_tools()

# Inspect tools
for tool in tools:
    print(tool.name, "=>", tool.description)

```

```

add => Add two numbers
multiply => Multiply two numbers

```

## Toolkit Usage

### 1. Create the Toolkit

```

toolkit = MathToolkit()

```

Creates an instance of the `MathToolkit` class.

### 2. Get All Tools

```

tools = toolkit.get_tools()

```

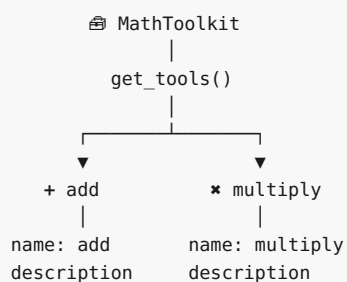
Retrieves all the tools available in the toolkit.

#### Output:

```

[add, multiply]

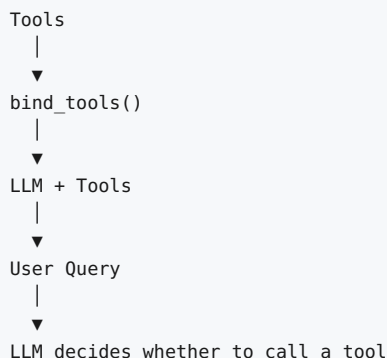
```



## Tool Binding

**Tool binding** is the process of connecting tools to an LLM so that the LLM **knows which tools are available and can decide when to call them**.

## Simple flow



## Example

```
#local ollama
from langchain_ollama import ChatOllama

model = ChatOllama(model="qwen3:4b")
response = model.invoke("the capital of India? in one word")
print(response.content)
```

Delhi

```
llm_with_tools = model.bind_tools(tools)
```

```
response = llm_with_tools.invoke("What is 10 multiplied by 5?")

print(response.tool_calls)
```

```
[{'name': 'multiply', 'args': {'a': 10, 'b': 5}, 'id': 'fced2dde-8e12-4468-a84c-28b4effae522', 'type': 'tool_call'}]
```

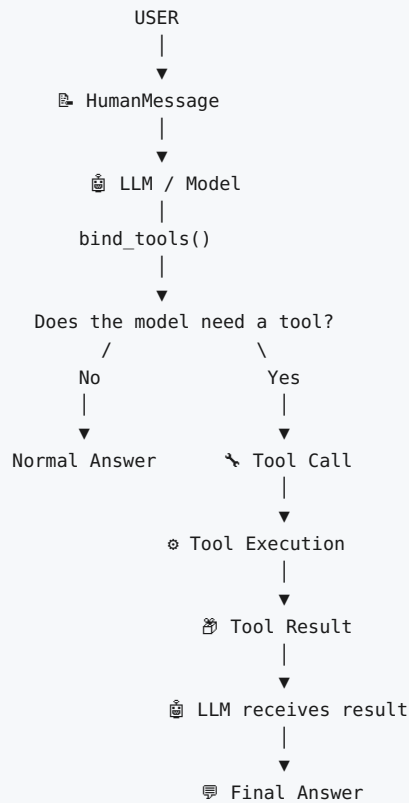
```
from langchain_core.messages import HumanMessage, ToolMessage
messages = [HumanMessage(content="Can you multiply 3 with 1000?")]
response = llm_with_tools.invoke(messages)
print(response.tool_calls[0])
result = multiply.invoke(response.tool_calls[0]["args"])
result
```

```
{'name': 'multiply', 'args': {'a': 3, 'b': 1000}, 'id': '6c9b6cef-f072-49fb-a0b2-c4503a9acb0b', 'type': 'tool_call'}
```

3000

## 🔗 Complete Flow of Tools in LangChain

The complete tool flow can be understood in **7 steps**:



## 1. Create a Tool

A **Tool** is a function that the LLM can request.

```
from langchain_core.tools import tool

@tool
def add(a: int, b: int) -> int:
    """Add two numbers"""
    return a + b

@tool
def multiply(a: int, b: int) -> int:
    """Multiply two numbers"""
    return a * b
```

We now have two tools:

```
✂ add
✂ multiply
```

---

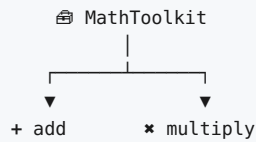
## 2. Create a Toolkit

A **Toolkit** groups related tools together.

```
class MathToolkit:
    def get_tools(self):
        return [add, multiply]

toolkit = MathToolkit()
```

Conceptually:



### 3. Get All Tools

```
tools = toolkit.get_tools()
```

The result is:

```
[add, multiply]
```

You can inspect the tools:

```
for tool in tools:
    print(tool.name, "=>", tool.description)
```

Output:

```
add => Add two numbers
multiply => Multiply two numbers
```

### 4. Create the LLM

Example using Ollama:

```
from langchain_ollama import ChatOllama

model = ChatOllama(
    model="qwen3:4b"
)
```

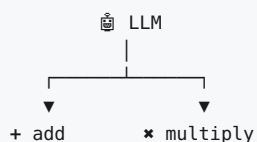
The selected Ollama model must support **tool calling**.

### 5. Bind Tools to the LLM

```
model_with_tools = model.bind_tools(tools)
```

This is called **Tool Binding**.

It tells the LLM which tools are available:



`bind_tools()` makes tools available to the LLM. It does **not** execute the tools.

### 6. Send the User Message

```
from langchain_core.messages import HumanMessage

messages = [
    HumanMessage(
        content="Can you multiply 3 with 1000?"
    )
]

response = model_with_tools.invoke(messages)
```

The LLM receives:

```
HumanMessage
|
▼
"Can you multiply 3 with 1000?"
```

---

## 7. LLM Generates a Tool Call

The LLM decides that it needs the `multiply` tool.

```
response.tool_calls
```

Example:

```
[
  {
    "name": "multiply",
    "args": {
      "a": 3,
      "b": 1000
    }
  }
]
```

The LLM is requesting:

```
multiply(a=3, b=1000)
```

The LLM **requests** the tool. It does not automatically mean your Python function has executed.

---

## 8. Execute the Tool

Get the tool call:

```
tool_call = response.tool_calls[0]
```

Execute the tool:

```
result = multiply.invoke(tool_call["args"])

print(result)
```

Output:

```
3000
```

Flow:

```
Tool Call
|
▼
multiply(a=3, b=1000)
|
▼
Python function executes
|
▼
3000
```

---

## 9. Send the Tool Result Back to the LLM

A `ToolMessage` sends the tool result back to the model.

```
from langchain_core.messages import ToolMessage

tool_message = ToolMessage(
    content=str(result),
    tool_call_id=tool_call["id"]
)
```

Then send the messages back to the LLM:

```
messages = messages + [
    response,
    tool_message
]

final_response = model_with_tools.invoke(messages)

print(final_response.content)
```

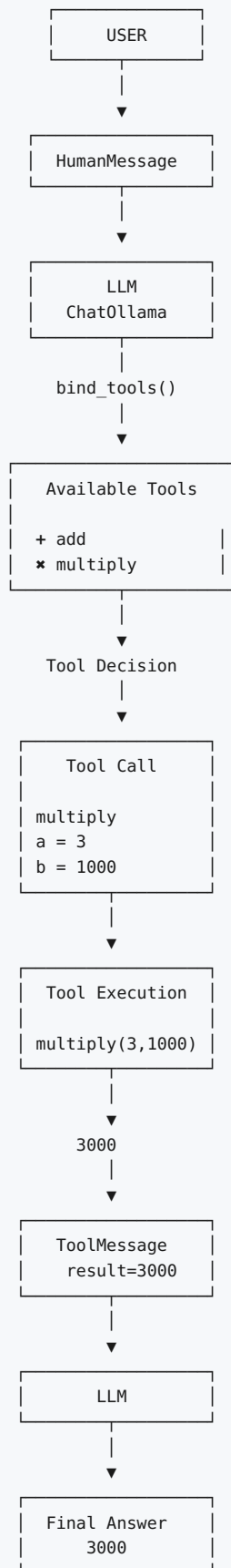
Final output:

```
3000
```

---

## Complete Tool Flow

---



## Key Concepts

| Concept | Meaning                        |
|---------|--------------------------------|
| Tool    | Individual function/capability |

|                       |                                       |
|-----------------------|---------------------------------------|
| <b>Toolkit</b>        | Collection of related tools           |
| <b>Tool Binding</b>   | Makes tools available to the LLM      |
| <b>Tool Call</b>      | LLM requests a specific tool          |
| <b>Tool Execution</b> | Application executes the tool         |
| <b>ToolMessage</b>    | Sends the tool result back to the LLM |

## ☆ Easy Way to Remember

```

User
↓
HumanMessage
↓
LLM
↓
Tool Binding
↓
Tool Call
↓
Tool Execution
↓
ToolMessage
↓
LLM
↓
Final Answer

```

```

# =====
# 1. IMPORTS
# =====

from langchain_core.tools import tool
from langchain_core.messages import HumanMessage, ToolMessage
from langchain_ollama import ChatOllama

# =====
# 2. CREATE CUSTOM TOOLS
# =====

@tool
def add(a: int, b: int) -> int:
    """Add two numbers"""
    return a + b

@tool
def multiply(a: int, b: int) -> int:
    """Multiply two numbers"""
    return a * b

# =====
# 3. CREATE TOOLKIT
# =====

class MathToolkit:

    def get_tools(self):
        return [add, multiply]

# Create toolkit
toolkit = MathToolkit()

```

```

# =====
# 4. GET ALL TOOLS FROM TOOLKIT
# =====

tools = toolkit.get_tools()

for tool in tools:
    print(tool.name, ">", tool.description)

# =====
# 5. CREATE OLLAMA MODEL
# =====

# IMPORTANT:
# Use an Ollama model that supports tool calling.
# gemma3:4b in your setup returned:
# "does not support tools"

model = ChatOllama(model="qwen3:4b")

# =====
# 6. BIND TOOLS TO MODEL
# =====

model_with_tools = model.bind_tools(tools)

# =====
# 7. CREATE HUMAN MESSAGE
# =====

messages = [HumanMessage(content="Can you multiply 3 with 1000?")]

# =====
# 8. INVOKE MODEL
# =====

response = model_with_tools.invoke(messages)

print("\nLLM Response:")
print(response)

# =====
# 9. CHECK WHETHER LLM REQUESTED A TOOL
# =====

print("\nTool Calls:")
print(response.tool_calls)

# =====
# 10. EXECUTE TOOL CALL
# =====

tool_messages = []

for tool_call in response.tool_calls:

    tool_name = tool_call["name"]
    tool_args = tool_call["args"]

    print("\nTool Name:", tool_name)
    print("Tool Arguments:", tool_args)

    # Find the requested tool
    selected_tool = next(
        tool for tool in tools
        if tool.name == tool_name
    )

```

```

# Execute the tool
result = selected_tool.invoke(tool_args)

print("Tool Result:", result)

# =====
# 11. CREATE TOOL MESSAGE
# =====

tool_message = ToolMessage(
    content=str(result),
    tool_call_id=tool_call["id"]
)

tool_messages.append(tool_message)

# =====
# 12. SEND TOOL RESULT BACK TO LLM
# =====

messages = messages + [response] + tool_messages

# =====
# 13. GET FINAL ANSWER
# =====

final_response = model_with_tools.invoke(messages)

print("\nFinal Answer:")
print(final_response.content)

```

add => Add two numbers  
multiply => Multiply two numbers

LLM Response:  
content='additional\_kwargs={} response\_metadata={'model': 'qwen3:4b', 'created\_at': '2026-09-26T13:50:10.466913Z', 'done': True, 'done\_reason': 'stop', 'total\_duration': 8769588209, 'load\_duration': 1050112792, 'prompt\_eval\_count': 186, 'prompt\_eval\_duration': 341448000, 'eval\_count': 338, 'eval\_duration': 7373545000, 'logprobs': None, 'model\_name': 'qwen3:4b', 'model\_provider': 'ollama'} id='lc\_run--01a0ddfa-e1e0-7113-8ff6-ef1eb07b1480-0' tool\_calls=[{'name': 'multiply', 'args': {'a': 3, 'b': 1000}, 'id': '0d7f5589-383c-41f4-a896-e8d4bd04f9c3', 'type': 'tool\_call'}] invalid\_tool\_calls=[] usage\_metadata={'input\_tokens': 186, 'output\_tokens': 338, 'total\_tokens': 524}'

Tool Calls:  
[{'name': 'multiply', 'args': {'a': 3, 'b': 1000}, 'id': '0d7f5589-383c-41f4-a896-e8d4bd04f9c3', 'type': 'tool\_call'}]

Tool Name: multiply  
Tool Arguments: {'a': 3, 'b': 1000}  
Tool Result: 3000

Final Answer:  
The result of multiplying 3 by 1000 is **\*\*3000\*\***.